

Documentation Technique — Vite & Gourmand

1. Réflexions initiales technologiques

Choix du langage back-end : PHP

Le choix de PHP s'est imposé naturellement pour plusieurs raisons :

- Langage enseigné dans le cadre de la formation Studi
- Très largement utilisé pour les applications web de type CRUD
- Compatible nativement avec Apache (XAMPP) sans configuration complexe
- Intégration simple avec MySQL via PDO
- Grande communauté et documentation abondante

Des alternatives comme Node.js ou Python (Django/Flask) ont été envisagées mais écartées car elles nécessitent une configuration plus complexe et s'éloignent du cadre pédagogique du projet.

Choix de la base de données : MySQL

MySQL a été choisi comme base de données relationnelle pour les raisons suivantes :

- Parfaitement adapté aux données structurées du projet (menus, commandes, utilisateurs)
- Compatible avec PDO pour des requêtes préparées sécurisées
- Outil phpMyAdmin disponible via XAMPP pour l'administration
- Schéma fourni dans le sujet basé sur MySQL

Choix du front-end : Bootstrap 5

Bootstrap 5 a été utilisé pour :

- Accélérer le développement de l'interface responsive
- Garantir une cohérence visuelle sur tous les écrans
- Fournir des composants prêts à l'emploi (navbar, cards, tabs, badges)
- Éviter de réinventer des solutions CSS complexes

Choix de PHPMailer

PHPMailer a été choisi pour la gestion des emails car :

- Bibliothèque PHP la plus utilisée pour l'envoi d'emails
- Support natif SMTP avec authentification Gmail
- Installation simple via Composer
- Support HTML dans les emails

2. Configuration de l'environnement de travail

Environnement local

- **Système d'exploitation** : Windows
- **Serveur web** : Apache via XAMPP
- **PHP** : 8.0.30
- **Base de données** : MySQL via XAMPP + phpMyAdmin
- **Éditeur de code** : Visual Studio Code
- **Gestionnaire de dépendances** : Composer 2.9.1
- **Versioning** : Git + GitHub

Installation des dépendances

bash

composer require phpmailer/phpmailer

Structure des branches Git

main

└─ develop

├─ feature/auth

└─ (autres features mergées)

3. Modèle Conceptuel de Données (MCD)

Le schéma de base de données fourni dans le sujet a été respecté. Les tables suivantes ont été implémentées :

Tables principales :

- utilisateur — stocke tous les comptes (utilisateur, employé, administrateur)
- role — définit les rôles (1: utilisateur, 2: employé, 3: administrateur)
- menu — les menus proposés par l'entreprise
- plat — les plats individuels
- commande — les commandes passées par les utilisateurs
- avis — les avis laissés par les utilisateurs après une commande
- horaire — les horaires d'ouverture affichés en pied de page
- regime — les régimes alimentaires (classique, végétarien, vegan)
- theme — les thèmes des menus (Noël, Pâques, classique, événement)
- allergene — les allergènes associés aux plats

Tables de jonction (many-to-many) :

- menu_plat — association entre menus et plats
- plat_allergene — association entre plats et allergènes

Relations clés :

- Un utilisateur possède un rôle (1,1 — 0,n)
- Un menu est associé à un régime et un thème
- Une commande appartient à un utilisateur et concerne un menu
- Un avis est lié à une commande et un utilisateur
- Un plat peut appartenir à plusieurs menus (many-to-many)
- Un plat peut contenir plusieurs allergènes (many-to-many)

4. Diagramme d'utilisation (Use Case)

Visiteur (non authentifié) :

- Consulter la page d'accueil
- Consulter les menus avec filtres
- Voir le détail d'un menu
- S'inscrire
- Se connecter
- Contacter l'entreprise

Utilisateur (authentifié) :

- Tout ce que le visiteur peut faire
- Passer une commande
- Consulter et gérer ses commandes
- Modifier ses informations personnelles
- Laisser un avis après une commande terminée

Employé :

- Tout ce que l'utilisateur peut faire
- Gérer les commandes (changer le statut)
- Valider ou refuser les avis clients
- Modifier et supprimer les menus et plats

Administrateur :

- Tout ce que l'employé peut faire
- Créer et désactiver des comptes employé
- Consulter les statistiques de commandes par menu
- Gérer l'ensemble de l'application

5. Sécurité

Authentification

- Les mots de passe sont hashés avec **bcrypt** via `password_hash()` et vérifiés avec `password_verify()`
- Les sessions PHP sont utilisées pour maintenir l'état de connexion
- Chaque page protégée vérifie la présence et le rôle de la session

Protection contre les injections SQL

- Toutes les requêtes utilisent des **requêtes préparées PDO** avec des paramètres liés
- Aucune interpolation directe de variables dans les requêtes SQL

Protection XSS

- Toutes les sorties utilisateur sont filtrées avec `htmlspecialchars()`

Validation des mots de passe

- Minimum 10 caractères
- Au moins une majuscule, une minuscule, un chiffre, un caractère spécial
- Validé côté serveur avec une expression régulière

Contrôle d'accès

- Vérification du rôle sur chaque page protégée
- Redirection vers la page de connexion si non authentifié
- Les actions critiques vérifient que la ressource appartient bien à l'utilisateur connecté

RGPD

- Les données personnelles collectées sont limitées au strict nécessaire
- Mentions légales et CGV disponibles sur le site
- Possibilité de modifier ses données personnelles depuis l'espace utilisateur

6. Déploiement

Tentatives de déploiement effectuées

Dans le cadre de ce projet, plusieurs plateformes de déploiement ont été testées. Malheureusement, l'écosystème des hébergements PHP gratuits en 2026 est très limité.

Plateforme	Résultat	Raison de l'échec
Railway	Échec	Erreur driver PDO MySQL — extensions PHP 8.4 non compatibles malgré configuration nixpacks
Fly.io	Échec	Aucun Dockerfile détecté, runtime PHP non reconnu automatiquement
Heroku	Non tenté	Plan gratuit supprimé définitivement depuis novembre 2022
000webhost	Non tenté	Service définitivement fermé
InfinityFree	Écarté	Pas de Remote MySQL, pas de support PHP mail, limitations critiques
Alwaysdata	Écarté	Validation par carte bancaire requise pour le compte gratuit

Configuration prévue pour la production

Le fichier config/db.php a été configuré pour détecter automatiquement l'environnement :

php

```
if(getenv('RAILWAY_ENVIRONMENT')) {  
    // Variables d'environnement Railway  
} else {  
    // Configuration locale XAMPP  
}
```

Accès local

L'application est entièrement fonctionnelle en local via XAMPP. La procédure d'installation est détaillée dans le fichier README.md du dépôt GitHub.

7. Gestion de projet

Méthodologie

Une approche **Agile simplifiée** a été adoptée, avec un tableau Trello organisé en trois colonnes : To Do, In Progress, Done. Chaque fonctionnalité a été développée de manière indépendante puis intégrée progressivement.

Versioning Git

Les bonnes pratiques Git ont été respectées :

- Une branche main pour la version stable
- Une branche develop pour le développement

- Des branches feature/ pour chaque fonctionnalité
- Des messages de commit descriptifs (feat:, fix:, docs:)